第十四讲教学设计：数据库事务与连接池 (20分钟)

课程信息

- **课程名称**: Java Web应用开发 (Java Web Application Development)
- **教学方式**: 双语教学（英文授课，关键术语中英文对照）
- **教学主题**: 数据库事务与连接池（事务控制、批处理、Druid基础）
- **授课时长**: 20分钟
- **适用对象**: 已完成三层与DAO重构、具备JDBC编程基础的学生

一、教学目标

知识目标

1. 理解事务（Transaction）的ACID特性与常见隔离级别（Isolation Level: READ COMMITTED/REPEATABLE READ等）。
2. 掌握JDBC事务控制（Transaction Control）：`setAutoCommit(false)`、`commit()`、`rollback()`、`Savepoint`。
3. 掌握批处理（Batch Processing）`addBatch()/executeBatch()` 的使用场景与注意事项。
4. 了解连接池（Connection Pool）的作用、基本参数与Druid的基础配置与使用方式。

能力目标

1. 能在Service层实现跨DAO的一致性操作（如“扣减库存 + 生成订单”）。
2. 能正确在异常情况下回滚并恢复连接状态，避免连接泄漏与脏连接。
3. 能引入Druid连接池并通过 `DataSource` 获取连接替代 `DriverManager`。

素质目标

1. 建立“数据一致性优先”的工程意识，避免“部分成功”导致的业务错乱。
2. 强化性能与稳定性意识：连接是昂贵资源，应池化复用并监控。
3. **[思政融合]** 以“诚信交易与契约精神”类比事务：要么全部达成要么全部撤销，维护公平与秩序。

二、教学重点与难点

教学重点

- Service层事务边界的定义与实现；统一连接的传递。
- 批处理在大批量场景的性能收益与参数控制。
- Druid连接池的最小可用配置与获取连接方式。

教学难点

- 在多个DAO方法中复用同一连接；避免DAO内部私自开关连接。
- 异常处理：回滚、重置 `autoCommit`、关闭资源的顺序与兜底。
- 隔离级别选择与并发异常（丢失更新/幻读）取舍。

三、教学内容与时间安排

第一部分：问题引入——“一半成功”的灾难 (4分钟)

内容要点

1. 订单创建需同时"扣减库存 + 写入订单表"。任一失败都应全部撤销。
2. 普通JDBC若每个DAO独立开关连接，容易出现部分提交，导致数据不一致。
3. 引出事务边界与连接池。

教学方法

- 场景叙述 + 失败案例剖析。

第二部分：新知讲解——事务控制与连接池 (10分钟)

内容要点

1. Service层事务模板：

```
public class OrderService {
    private final OrderDao orderDao = new JdbcOrderDao();
    private final InventoryDao inventoryDao = new JdbcInventoryDao();

    public void placeOrder(Long userId, Long productId, int qty) {
        Connection conn = null;
        try {
            conn = DataSourceUtil.getConnection(); // 来自连接池
            conn.setAutoCommit(false);              // 1) 开启事务
            conn.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);

            inventoryDao.decrease(conn, productId, qty);   // 2) 扣减库存
            Long orderId = orderDao.insert(conn, userId, productId, qty); // 3) 写
订单

            conn.commit();                          // 4) 提交
        } catch (Exception e) {
            if (conn != null) try { conn.rollback(); } catch (SQLException ignore)
{}
            throw new RuntimeException("下单失败，已回滚", e);
        } finally {
            if (conn != null) {
                try { conn.setAutoCommit(true); } catch (SQLException ignore) {}
                try { conn.close(); } catch (SQLException ignore) {}
            }
        }
    }
}
```

2. DAO层接受外部连接：

```
public class JdbcInventoryDao implements InventoryDao {
    @Override
    public void decrease(Connection conn, Long productId, int qty) throws
```

```
SQLException {
        String sql = "UPDATE t_product SET stock = stock - ? WHERE id = ? AND stock
>= ?";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, qty);
            ps.setLong(2, productId);
            ps.setInt(3, qty);
            if (ps.executeUpdate() == 0) throw new SQLException("库存不足");
        }
    }
}

public class JdbcOrderDao implements OrderDao {
    @Override
    public Long insert(Connection conn, Long userId, Long productId, int qty)
throws SQLException {
        String sql = "INSERT INTO t_order(user_id, created_at) VALUES (?, now())";
        try (PreparedStatement ps = conn.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {
            ps.setLong(1, userId);
            ps.executeUpdate();
            try (ResultSet keys = ps.getGeneratedKeys()) {
                if (keys.next()) {
                    long orderId = keys.getLong(1);
                    insertItemBatch(conn, orderId, productId, qty);
                    return orderId;
                }
            }
        }
        throw new SQLException("插入订单失败");
    }
    private void insertItemBatch(Connection conn, long orderId, Long productId, int
qty) throws SQLException {
        try (PreparedStatement ps = conn.prepareStatement(
                "INSERT INTO t_order_item(order_id, product_id, qty) VALUES
(?,?,?)")) {
            // 演示批处理：此处仅一行，实际可循环多行
            ps.setLong(1, orderId); ps.setLong(2, productId); ps.setInt(3, qty);
ps.addBatch();
            ps.executeBatch();
        }
    }
}
```

3. Druid连接池最小可用配置（示例）：

```
# druid.properties
driverClassName=com.mysql.cj.jdbc.Driver
url=jdbc:mysql://localhost:3306/demo?
useSSL=false&serverTimezone=UTC&characterEncoding=UTF-8
username=root
password=123456
initialSize=5
maxActive=20
minIdle=5
maxWait=60000
```

4. 获取连接的工具：

```
import com.alibaba.druid.pool.DruidDataSourceFactory;
import javax.sql.DataSource;
public class DataSourceUtil {
    private static DataSource ds;
    static {
        try {
            Properties p = new Properties();
            try (InputStream in =
DataSourceUtil.class.getClassLoader().getResourceAsStream("druid.properties")) {
                p.load(in);
            }
            ds = DruidDataSourceFactory.createDataSource(p);
        } catch (Exception e) {
            throw new ExceptionInInitializerError(e);
        }
    }
    public static Connection getConnection() throws SQLException { return
ds.getConnection(); }
    public static DataSource getDataSource() { return ds; }
}
```

5. 可选：保存点与回滚片段

```
Savepoint sp = conn.setSavepoint();
// ... 执行部分操作
conn.rollback(sp); // 回滚到保存点
```

教学方法

- 代码走查 + 时序图（开启→执行→提交/回滚）+ 小实验（制造异常验证回滚）。

第三部分：实战——下单流程的一致性验证 (4分钟)

内容要点

1. 故意在扣减库存后抛出异常，观察是否已成功回滚。
2. 增加批量插入订单项，观察批处理对性能的影响（以日志时间对比）。
3. 使用连接池并对比冷启动与复用连接的响应时间差。

教学方法

- 现场联调 + 控制变量实验。

第四部分：课堂总结与展望 (2分钟)

总结要点

1. 事务边界应在Service层，DAO不私自开关连接；异常要回滚并复位连接状态。
2. 连接池是生产标配；合理配置能显著提升吞吐与稳定性。

展望（承上启下）

- 下一讲将把“安全与治理”向前推进：实现用户认证与基于角色的权限控制。

四、教学方法与手段

教学方法

1. **问题驱动 (Problem-Driven)**: 从"一半成功"灾难场景引入
2. **代码走查 (Code Walkthrough)**: 讲解Service层事务模板
3. **实验验证 (Experiment)**: 制造异常验证回滚机制
4. **对比实验 (Comparison)**: 批处理与连接池性能对比
5. **双语教学法 (Bilingual Teaching)**:
    - 课堂采用英文授课，关键技术术语提供中英文对照
    - 重要概念（Transaction, ACID, Isolation Level, Batch Processing, Connection Pool）首次出现时给出英文表达和中文释义
    - 鼓励学生使用英文术语描述事务和连接池
    - 代码注释和变量命名使用规范英文

教学手段

- IDE (IntelliJ IDEA/Eclipse)
- Tomcat服务器 (Tomcat Server)
- 数据库客户端 (Database Client)
- PPT/Marp课件 (Presentation Slides)

五、教学评价

- **过程性评价**: 能否说出事务四要素与隔离级别选用理由；能否解释批处理的优势与风险。
- **终结性评价**: 能否实现一个跨DAO的事务性操作并在异常时正确回滚；能否接入Druid并通过 `DataSource` 获取连接。

六、课后拓展

思考题

1. 为什么要在 `finally` 中将 `autoCommit` 设回 `true` ？如果忘记，会出现什么隐患？

2. 生产环境如何选择隔离级别？面对高并发下幻读/丢失更新，你会如何权衡？

实践作业

1. 将“下单流程”抽象为事务模板方法，统一异常与回滚处理；
2. 在 `druid.properties` 中开启SQL监控，收集并分析慢SQL，给出优化建议。